

Trabalho Prático 2 – Gerenciador de Memória Virtual

Disciplina: Sistemas Operacionais

Curso: Sistemas de Informação – PUC Minas

Aluno: Guilherme Pimentel

Professor: *Lucas Bragança da Silva*

1. Introdução

O gerenciamento de memória é uma das funções mais importantes de um sistema operacional, sendo responsável por controlar a utilização da memória principal e permitir que vários processos sejam executados de forma eficiente. Uma das técnicas utilizadas para isso é a memória virtual, que possibilita que programas utilizem um espaço de endereçamento maior do que a memória física disponível.

Neste trabalho foi desenvolvido um simulador de gerenciamento de memória virtual em linguagem C, utilizando os conceitos estudados em sala de aula, como paginação por demanda, tabela de páginas, Translation Lookaside Buffer (TLB), tratamento de Page Fault e substituição de páginas utilizando uma aproximação do algoritmo Least Recently Used (LRU).

O objetivo foi compreender o funcionamento interno da tradução de endereços e observar como diferentes padrões de acesso influenciam o desempenho do sistema.

2. Fundamentação Teórica

Memória Virtual

A memória virtual permite que cada processo utilize um espaço de endereçamento lógico independente da quantidade de memória física instalada no computador.

Quando uma página ainda não está carregada na memória principal, ocorre uma falta de página (Page Fault), sendo necessário buscá-la no armazenamento secundário.

Paginação

A memória virtual é dividida em páginas de tamanho fixo, enquanto a memória física é dividida em quadros (frames) do mesmo tamanho.

Neste trabalho foram utilizados:

- Tamanho da página: 256 bytes
- Número de páginas: 256
- Número de quadros: 128

Como existem mais páginas do que quadros, nem todas podem permanecer carregadas simultaneamente.

Tabela de Páginas

A tabela de páginas mantém o mapeamento entre páginas virtuais e quadros físicos.

Cada entrada possui:

- Número do quadro;
- Bit de validade;
- Bit de referência;
- Contador de envelhecimento (Aging).

TLB

O Translation Lookaside Buffer (TLB) é uma pequena memória cache utilizada para acelerar a tradução de endereços.

Quando uma página é encontrada no TLB ocorre um **TLB Hit**, evitando consultar a tabela de páginas.

Neste projeto o TLB possui:

- 16 entradas;
- Política FIFO para substituição.

Algoritmo Aging (LRU Aproximado)

Como o algoritmo LRU puro possui alto custo computacional, foi utilizada a técnica Aging.

Cada página possui:

- Bit de referência;
- Contador de envelhecimento de 8 bits.

Após cada acesso:

- o contador é deslocado para a direita;
- o bit de referência é inserido no bit mais significativo;
- o bit de referência é zerado.

A página com menor contador é escolhida para substituição.

3. Desenvolvimento

O simulador foi desenvolvido utilizando os arquivos disponibilizados pelo professor, implementando os trechos marcados com TODO.

As principais funcionalidades implementadas foram:

- separação do endereço lógico em número da página e deslocamento (offset);
- consulta ao TLB;
- consulta à tabela de páginas;
- tratamento de Page Fault;
- carregamento de páginas do arquivo BACKING_STORE.bin;
- atualização da memória física;
- atualização da tabela de páginas;
- substituição de páginas utilizando Aging;
- atualização do TLB utilizando FIFO;
- cálculo das estatísticas finais.

Durante a execução, cada endereço lógico é traduzido para um endereço físico e o valor armazenado é apresentado na tela.

4. Estruturas Utilizadas

Memória Física

A memória física foi implementada utilizando um vetor bidimensional:

```
signed char physical_memory[NUM_FRAMES][FRAME_SIZE];
```

Cada linha representa um quadro físico.

Tabela de Páginas

Cada entrada da tabela contém:

- quadro físico;
- bit de validade;
- bit de referência;
- contador Aging.

TLB

O TLB foi implementado utilizando um vetor de estruturas contendo:

- página;
- quadro;
- bit de validade.

A substituição ocorre utilizando FIFO.

5. Funcionamento do Programa

O programa executa os seguintes passos para cada endereço lógico:

1. Extrai o número da página e o offset.
2. Procura a página no TLB.
3. Caso não encontre, consulta a tabela de páginas.
4. Se a página não estiver carregada ocorre um Page Fault.
5. A página é carregada do BACKING_STORE.bin.
6. A tabela de páginas e o TLB são atualizados.
7. Calcula-se o endereço físico.
8. O valor armazenado é lido e exibido.

Ao final são apresentadas as estatísticas de execução.

6. Resultados Obtidos

Foram realizados dois testes utilizando os arquivos fornecidos.

Teste 1 – Endereços Aleatórios

Resultados obtidos:

Number of Translated Addresses = 10000

Page Faults = 5051

Page Fault Rate = 0.505

TLB Hits = 629

TLB Hit Rate = 0.063

Como os endereços foram gerados de forma aleatória, houve pouca reutilização das páginas carregadas. Consequentemente, aproximadamente metade dos acessos provocou Page Fault e apenas 6,3% das consultas resultaram em TLB Hit.

Teste 2 – Localidade de Referência

Resultados obtidos:

Number of Translated Addresses = 10000

Page Faults = 1019

Page Fault Rate = 0.102

TLB Hits = 7981

TLB Hit Rate = 0.798

Neste teste os endereços apresentam localidade de referência, ou seja, várias páginas próximas são acessadas repetidamente.

Esse comportamento reduziu significativamente a quantidade de Page Faults e aumentou a taxa de acertos do TLB para aproximadamente 80%.

Esses resultados demonstram claramente a eficiência da memória virtual quando existe reutilização das páginas carregadas.

7. Dificuldades Encontradas

As maiores dificuldades durante o desenvolvimento foram compreender o funcionamento da memória virtual, implementar corretamente o algoritmo Aging e integrar a comunicação entre a tabela de páginas, o TLB e a memória física.

Também foi necessário configurar o ambiente de desenvolvimento em C no Windows, incluindo a instalação do compilador GCC e geração dos arquivos de entrada utilizados nos testes.

8. Uso de Inteligência Artificial

Durante o desenvolvimento foi utilizada a ferramenta ChatGPT como apoio para esclarecer dúvidas sobre os conceitos de memória virtual, paginação, TLB, algoritmo Aging e linguagem C.

A ferramenta também auxiliou na compreensão da estrutura do projeto, na interpretação do código-base disponibilizado pelo professor e na implementação das funções marcadas como TODO.

Prompts utilizados no decorrer do trabalho:

Estou desenvolvendo um simulador de gerenciamento de memória virtual em linguagem C para a disciplina de Sistemas Operacionais. O projeto utiliza paginação, tabela de páginas, TLB e memória física. Gostaria que você explicasse, passo a passo, como ocorre a tradução de um endereço lógico para um endereço físico, mostrando o caminho percorrido desde a leitura do endereço até a obtenção do valor armazenado na memória. Se possível, utilize um exemplo numérico simples para facilitar o entendimento.

Meu projeto utiliza páginas de 256 bytes e endereços lógicos de 16 bits. Gostaria de confirmar se estou separando corretamente o número da página e o deslocamento (offset). Explique como realizar essa separação utilizando operações de deslocamento de bits e máscaras em linguagem C, mostrando também por que essas operações funcionam.

Estou implementando a tabela de páginas do simulador. Cada entrada possui número do quadro, bit de validade, bit de referência e contador de envelhecimento. Analise essa estrutura e explique qual é a função de cada campo dentro do gerenciamento de memória virtual, além de comentar como eles interagem durante um acesso à memória.

Preciso implementar o tratamento de Page Fault no meu projeto. Explique detalhadamente quais passos devem acontecer quando uma página não é encontrada na memória física. Gostaria de entender desde a identificação da falta de página até a atualização da memória, da tabela de páginas e do TLB, sem simplesmente fornecer o código pronto.

Meu projeto utiliza o algoritmo Aging como aproximação do LRU para substituição de páginas. Explique detalhadamente como funciona esse algoritmo, como o contador de envelhecimento deve ser atualizado após cada acesso e como a página vítima é escolhida quando não existem mais quadros livres na memória física.

Estou implementando um TLB com política FIFO. Gostaria que você revisasse a lógica que estou utilizando para inserção, atualização e remoção de entradas. Explique em quais situações ocorre um TLB Hit, quando deve ser realizada uma consulta à tabela de páginas e como evitar inconsistências quando uma página é removida da memória física.

Após implementar o simulador, obtive os seguintes resultados: aproximadamente 50% de Page Faults para acessos aleatórios e cerca de 10% para acessos com localidade de referência, além de um aumento significativo na taxa de acertos do TLB. Explique por que esse comportamento é esperado do ponto de vista teórico e relacione esses resultados com os conceitos de localidade espacial e temporal estudados em Sistemas Operacionais.

Estou desenvolvendo este trabalho utilizando GCC no Windows e encontrei dificuldades para configurar corretamente o ambiente de desenvolvimento no Visual Studio Code. Explique quais ferramentas são necessárias para compilar projetos em linguagem C utilizando Makefile e como verificar se o compilador e as variáveis de ambiente foram configurados corretamente.

Gostaria que você revisasse algumas funções do meu código em C relacionadas à tabela de páginas e ao gerenciamento da memória física. Em vez de reescrever o código completo, explique se a lógica utilizada está correta, aponte possíveis melhorias e destaque boas práticas de programação que poderiam ser aplicadas nesse tipo de projeto.

Estou finalizando o trabalho e preciso elaborar o relatório técnico. Com base no funcionamento do simulador de memória virtual, sugira uma estrutura adequada para o relatório, indicando quais conceitos devem ser apresentados na fundamentação teórica, quais decisões de implementação merecem destaque e como interpretar os resultados obtidos nos testes realizados.

9. Conclusão

O desenvolvimento deste trabalho permitiu compreender, na prática, como funciona o gerenciamento de memória virtual realizado pelos sistemas operacionais.

Foi possível observar o papel da tabela de páginas, do TLB, do tratamento de faltas de página e do algoritmo de substituição de páginas. Os testes mostraram claramente que programas com localidade de referência apresentam menor quantidade de Page Faults e maior taxa de acertos no TLB, melhorando significativamente o desempenho da tradução de endereços.

Dessa forma, o trabalho contribuiu para consolidar os principais conceitos estudados na disciplina de Sistemas Operacionais e demonstrou a importância das técnicas utilizadas pelos sistemas modernos para otimizar o uso da memória.

10. Referências

- SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. **Fundamentos de Sistemas Operacionais**. 10. ed. LTC, 2018.
- TANENBAUM, Andrew S.; BOS, Herbert. **Sistemas Operacionais Modernos**. 4. ed. Pearson, 2016.
- PUC Minas. **Trabalho Prático 2 – Gerenciador de Memória Virtual**. Material disponibilizado pelo professor da disciplina de Sistemas Operacionais, 2026.